

Table of Contents

<u>Sl.no.</u>	<u>Title</u>	<u>Page No.</u>
1	Introduction	3
2	Problem Statement	3
3	Objectives	3
4	Mathematical Approach	4
5	Data Source	4
6	Algorithm Design	4
7	Program Implementation	5
8	Simulation	5
9	Conclusion	5
10	Future Scope	6
11	Refference	6

1. Introduction -

Queueing systems play a crucial role in many real-world environments, including canteens, banks, hospitals, and transportation networks. In such systems, customers or entities arrive to receive service, often leading to waiting lines when demand exceeds immediate service capacity. Effective management of these queues is essential for minimizing waiting times, improving operational efficiency, and enhancing overall service quality.

This project focuses on the analysis of queue behavior using the M/M/6 queue model, a widely used multi-server queueing framework in operations research. In this model, arrivals are assumed to follow a Poisson distribution, while service times are exponentially distributed. The system consists of multiple servers (six in this case) that operate simultaneously to handle incoming customers, allowing for improved service throughput compared to single-server systems.

2. Problem Statement -

Manual analysis of queueing systems is often complex, time-consuming, and inefficient, particularly in dynamic environments where system conditions frequently change. In many real-world scenarios, customer arrival patterns vary over time, making it difficult to accurately predict demand and allocate resources effectively. The presence of multiple servers further increases the complexity of analysis, as interactions between servers and fluctuating workloads must be considered simultaneously. Additionally, the need for real-time decision-making adds another layer of difficulty, requiring continuous monitoring and quick interpretation of system behavior.

Several key challenges arise in the manual management of queueing systems. These include accurately estimating arrival rates and service rates, identifying peak load periods, and evaluating overall system efficiency. Furthermore, visualizing queue performance in a meaningful and interpretable way remains a significant challenge, especially when dealing with large volumes of data.

3. Objectives -

The main objective of this project is to analyze and enhance the performance of queueing systems using a data-driven approach. In many real-world scenarios, queueing systems experience issues such as variable customer arrivals, long waiting times, and inefficient utilization of resources. These challenges make it necessary to study queue behavior systematically.

This project focuses on collecting and analyzing queue data to understand system dynamics. Key parameters, including the arrival rate (λ) and service rate (μ), are calculated to evaluate customer flow and service efficiency. The level of system utilization is also determined to assess how effectively multiple servers are being used. In addition, visualization techniques such as charts and heatmaps are applied to represent data clearly, helping to identify peak and low load periods.

4. Mathematical Approach

This project utilizes the M/M/6 queue model, a multi-server queueing system used to analyze situations where multiple service channels operate simultaneously. In this model, customer arrivals follow a Poisson distribution, and service times follow an exponential distribution, making it suitable for systems with random and independent events.

The arrival rate, denoted by λ (lambda), represents the average number of customers arriving per minute, while the service rate, μ (mu), indicates the service capacity of each server. The system consists of six servers ($s = 6$), allowing multiple customers to be served at the same time. This model helps in evaluating system performance and understanding queue behavior under varying conditions.

5. Data Source

The system is based on real observed data collected from a canteen environment during peak operational hours, specifically between 12:00 PM and 2:00 PM. The dataset includes time-based records of the number of people present in the queue, along with a constant number of servers (six) available throughout the observation period. This real-world data provides a practical foundation for analyzing queue behavior and system performance.

The collected data is stored in CSV format using Google Sheets and is dynamically fetched into the web-based system. This approach enables real-time data updates, ensures scalability, and supports accurate and efficient analysis. By integrating live data with the analytical model, the system can provide meaningful insights and reflect actual queue conditions effectively.

6. Algorithm design

The system follows a structured algorithm to analyze queue data efficiently. Initially, input data consisting of time slots and the number of people present is collected and converted into a suitable numerical format for processing. Based on this data, key parameters such as the arrival rate (λ) and total service rate (μ_{total}) are computed to represent customer inflow and overall service capacity.

Using these values, the system calculates utilization (ρ) to evaluate the load on the queueing system. Based on the utilization level, the system is classified into low, moderate, or high load conditions. Further analysis is performed to identify peak and lowest load periods, and the results are presented using charts and heatmaps. This approach enables clear visualization and helps in understanding system performance effectively.

7. Program Implementation

The system is implemented using a modular approach to ensure clarity, scalability, and efficient processing. The Data Processing Module is responsible for fetching queue data from a CSV source and converting it into a numerical format suitable for analysis. This processed data is then passed to the Computation Module, where key parameters such as arrival rate (λ), service rate (μ), and system utilization (ρ) are calculated. Based on these computations, the system also generates meaningful insights regarding queue performance.

The Visualization Module presents the analyzed data using various graphical representations, including line charts for utilization and arrival rate, bar charts for crowd distribution, and heatmaps to illustrate system load patterns. Finally, the User Interface (UI) Module integrates all components into an interactive dashboard, providing a structured layout and a dedicated insights page. This modular design ensures that the system remains user-friendly while effectively communicating analytical results.

8. Simulation

Programming Language : JavaScript / TypeScript
Frontend : React.js and Next.js
Styling : Tailwind CSS
Visualization : Recharts for charts and heatmaps
Backend / Data Source : Google Sheets (CSV API for dynamic data fetching)
Development Platform : Visual Studio Code
Hosting Platform : Vercel

9. Conclusion

Result:
Successfully analyzed queue data
Identified peak load time (12:30 PM)
Calculated system utilization (~0.59)
Built interactive dashboard

Conclusion:
The system provides:
Accurate queue analysis
Clear visualization
Efficient performance tracking
It demonstrates practical application of Queueing Theory in real-world systems.

10. Future Scope

The proposed system can be further enhanced by integrating real-time data collection through IoT sensors to improve accuracy and responsiveness, as well as extending its capability to support multi-location queue tracking for centralized monitoring. Incorporating Machine Learning techniques can enable predictive analytics for forecasting customer flow and optimizing resource allocation, while developing a mobile application can enhance accessibility and user convenience. The system also has wide applicability across domains such as canteens and cafeterias, banks and hospitals, transportation systems, and event crowd management, where it can help reduce waiting times, improve service efficiency, and support better decision-making.

11. Reference

Queueing Theory – Operations Research textbooks
GeeksforGeeks (Queue Models)
Recharts Documentation
React & Next.js Docs
Google Sheets API